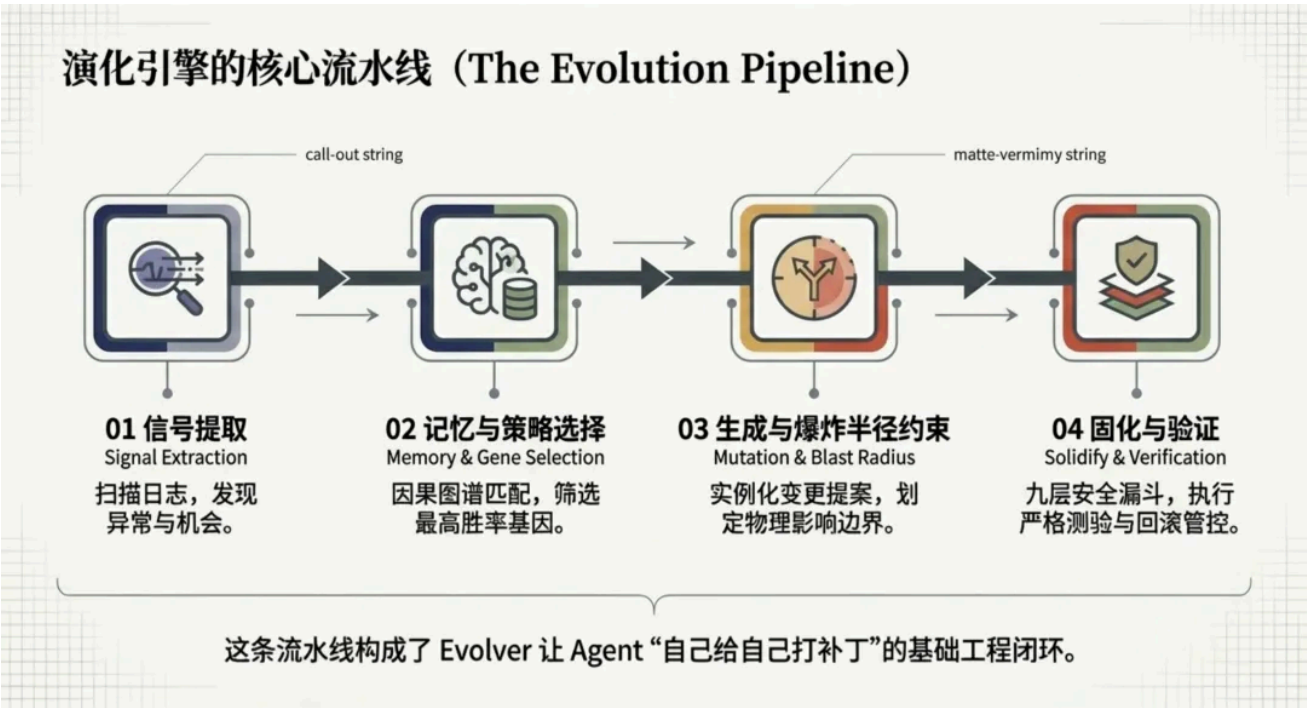


Evolver 拆解：一个让 Agent 自己演化自己的 Harness

原创 克熵Sam 航天智人 2026年3月17日 09:00 河北

AlphaEvolve 用进化算法改进代码，发现了 56 年来对 Strassen 矩阵乘法算法的首次改进。但 AlphaEvolve 是用来改进数学算法的，不是用来改进 Agent 自身的。

如果一个 Agent 能用类似的思路，审查自己的运行日志，发现自己的错误模式，然后自动给自己打补丁——这件事能工程化吗？

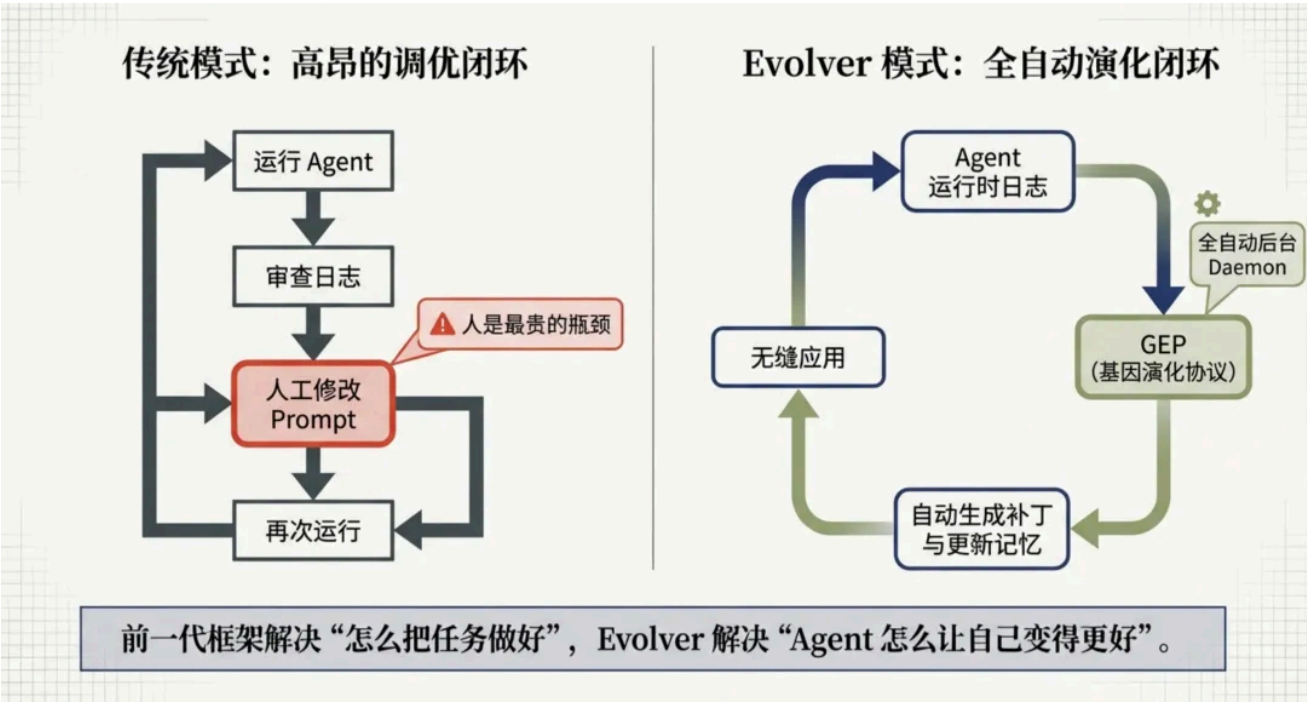


EvoMap/evolver 给出了一个完整的尝试。它不是一个 Agent 框架，而是一个 **协议约束的自演化引擎**：Agent 检查自己的运行历史，提取信号，选择策略，生成代码补丁，验证变更，然后把成功的经验沉淀成可复用的资产。整个过程有审计轨迹，有回滚机制，有爆炸半径控制。

前三篇我拆的 OpenClaw、Codex、MimiClaw 都在解同一类问题：怎么让 Agent 把当前任务做好。Evolver 问的是下一个问题：Agent 怎么让自己变得更好？

Evolver 在解什么题

大多数 Agent 的 prompt 调优流程长这样：跑一圈 → 看日志 → 人工改 prompt → 再跑一圈。这个循环里最贵的是人。



Evolver 要自动化这个循环。它的输入是 Agent 的运行日志、会话记录、内存文件；输出是代码补丁、新的技能定义、更新后的记忆。中间的决策过程被一个叫 GEP（Genome Evolution Protocol）的协议严格约束。

GEP 借了生物学的隐喻，但别被名字骗了——底层是一套严格的 JSON schema 和状态机：

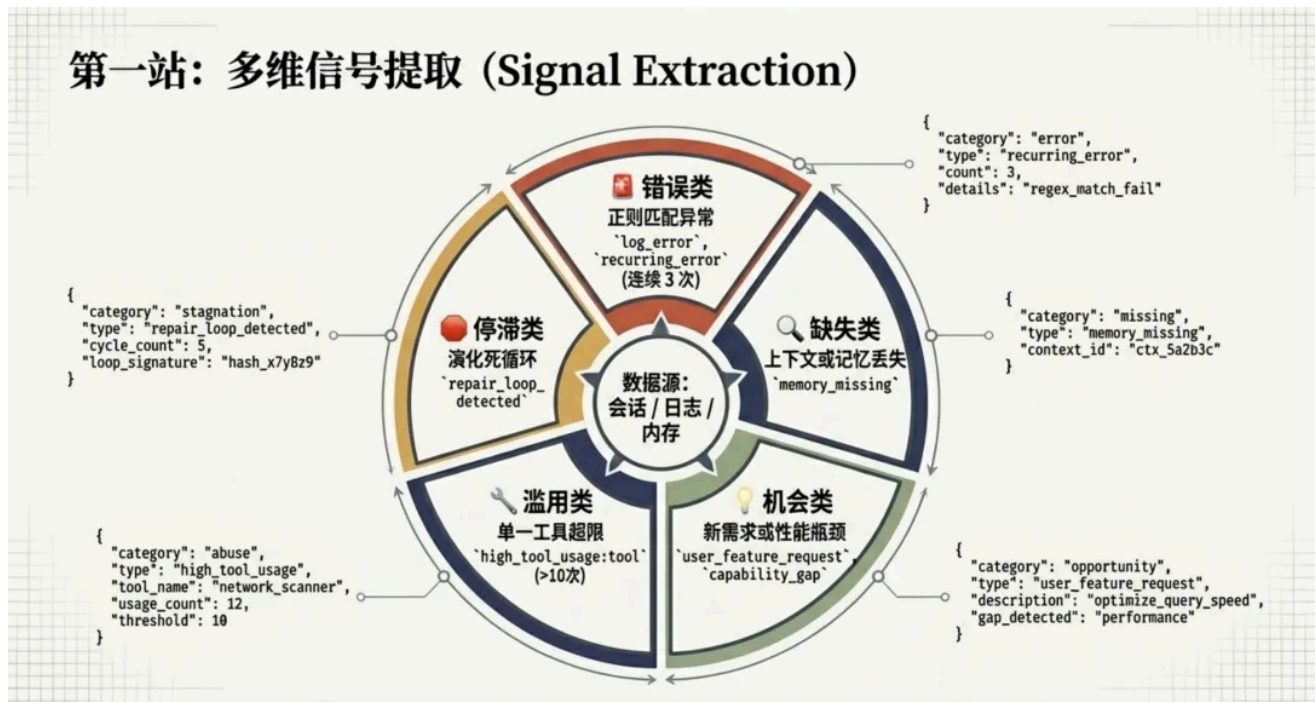
GEP 协议：祛魅生物学隐喻

生物学隐喻	工程实体转译	代码片段 / JSON 定义
Gene（基因）	策略模板 ：一组可复用的修复/优化策略，内嵌前置条件、约束与安全验证命令。	<pre>{ "max_files": 3, "validation_cmd": "npm test"}</pre>
Capsule（胶囊）	执行记录 ：Gene 的成功快照，包含触发信号、结果摘要、置信度与爆炸半径。	<pre>{ "signal_match": true, "confidence": 0.92}</pre>
Mutation（变异）	变更提案 ：当前 Cycle 的具体修改意图（Repair/Optimize/Innovate）、目标及风险等级。	<pre>{ "type": "repair", "risk_level": "low"}</pre>
EvolutionEvent（演化事件）	审计日志 ：完整的 events.jsonl，记录每次演化 cycle 的绝对轨迹。	<pre>{ "timestamp": 171500, "event": "solidified"}</pre>

底层并非魔法，而是一套极度严格的 JSON Schema 与状态机。

拆解一：信号提取与反停滞机制

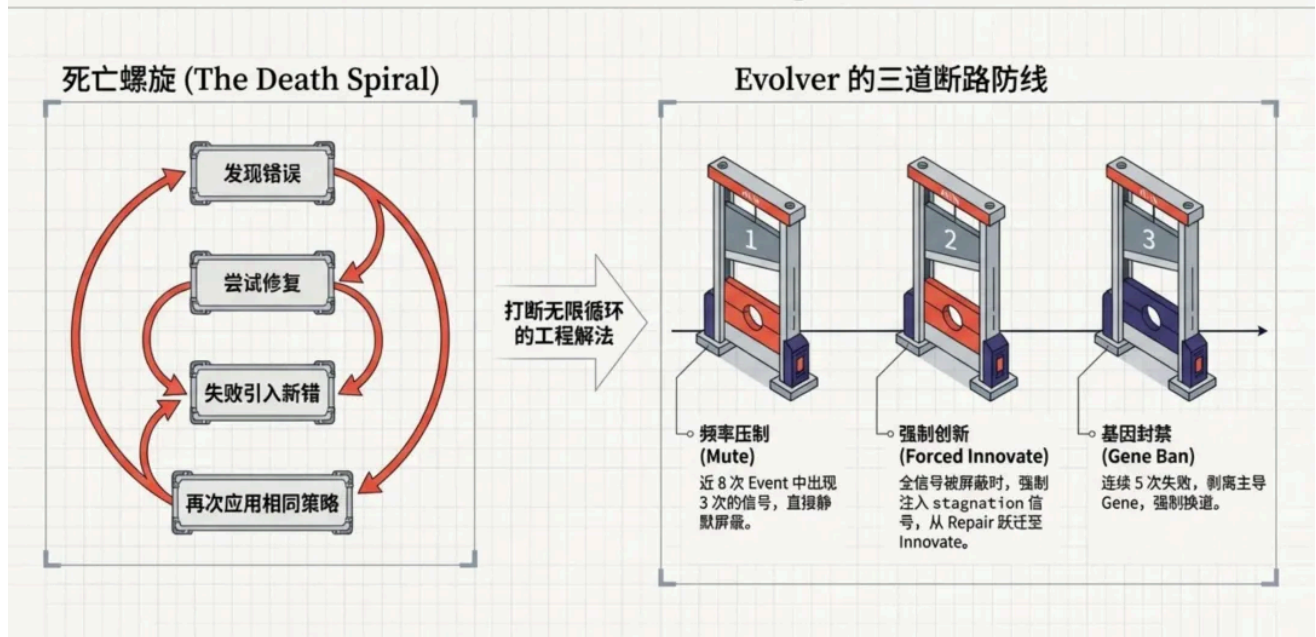
每个演化 cycle 从信号提取开始。extractSignals() 扫描最近的会话日志、今日日志、内存文件和历史演化事件，用正则匹配和统计分析提取信号。



信号的种类比我预期的丰富：

- 错误类：log_error、errsig:<text>、recurring_error（同一错误出现 3 次以上）
- 缺失类：memory_missing、user_missing、session_logs_missing
- 机会类：user_feature_request、capability_gap、perf_bottleneck
- 工具滥用：high_tool_usage:<tool>（某工具调用超 10 次）、repeated_tool_usage:exec
- 停滞类：repair_loop_detected、evolution_stagnation_detected、empty_cycle_loop_detected

反停滞断路器：打破死亡螺旋 (Anti-Stagnation Mechanism)



有意思的是反停滞设计。系统会检查最近 10 次 EvolutionEvent，如果一个信号在最近 8 次里出现了 3 次以上，直接压制这个信号——不让系统在同一个问题上反复打转。如果所有信号都被压制了，会强制注入 `evolution_stagnation_detected` 和 `stable_success_plateau`，迫使系统从 `repair` 跳到 `innovate`。

连续 5 次失败后，占主导地位的 Gene 的匹配信号会被剥离，强制选一个不同的 Gene。

这解决了自演化系统最容易掉进去的陷阱：修复循环。Agent 发现一个错误 → 尝试修复 → 修复失败引入新错误 → 再次检测到错误 → 再次尝试相同策略。Evolver 用信号频率统计 + 强制创新切换打断这个循环。

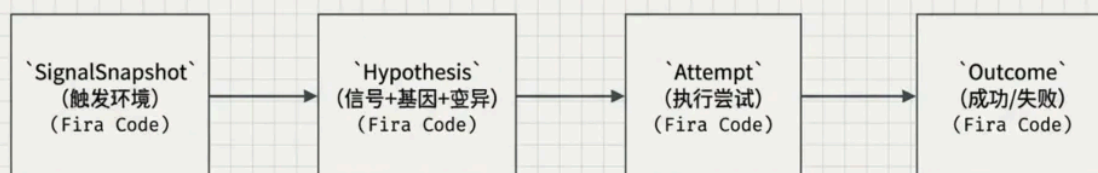
拆解二：记忆图谱的因果推理

Evolver 有一个记忆图谱系统存在 `memory/graph/` 目录下，记录四种节点：

`SignalSnapshot` → `Hypothesis` → `Attempt` → `Outcome`

每次 cycle 开始时，先关闭上一次 Attempt 的 Outcome（成功还是失败），然后记录当前的 `SignalSnapshot`，再生成 `Hypothesis`（信号 + 选中的 Gene + Mutation）。

第二站：策略级因果学习 (Causal Memory Graph)



策略筛选机制 (Gene Selection Dashboard)	
🚫 本局封禁 (Local Ban)	尝试 > 2次 且 成功率 < 0.18
🚫 永久封禁 (Global Ban)	尝试 > 3次 且 全局成功率 < 0.12
✅ 优先推荐 (Preferred)	当前节点胜率最高者

核心洞察：这是唯一的“策略级因果学习”设计。记录不仅是为了保全上下文，而是为了知道“为什么失败”。不踩同样的坑。

这个图谱的核心价值在 `getMemoryAdvice()`：它分析历史 Hypothesis → Attempt → Outcome 链条，计算每个 Gene 的成功率。成功率低于 0.18 且尝试过 2 次以上的 Gene 会被 ban 掉；全局成功率低于 0.12 且尝试过 3 次以上的 Gene 也会被 ban。成功率最高的 Gene 会被标记为 preferred。

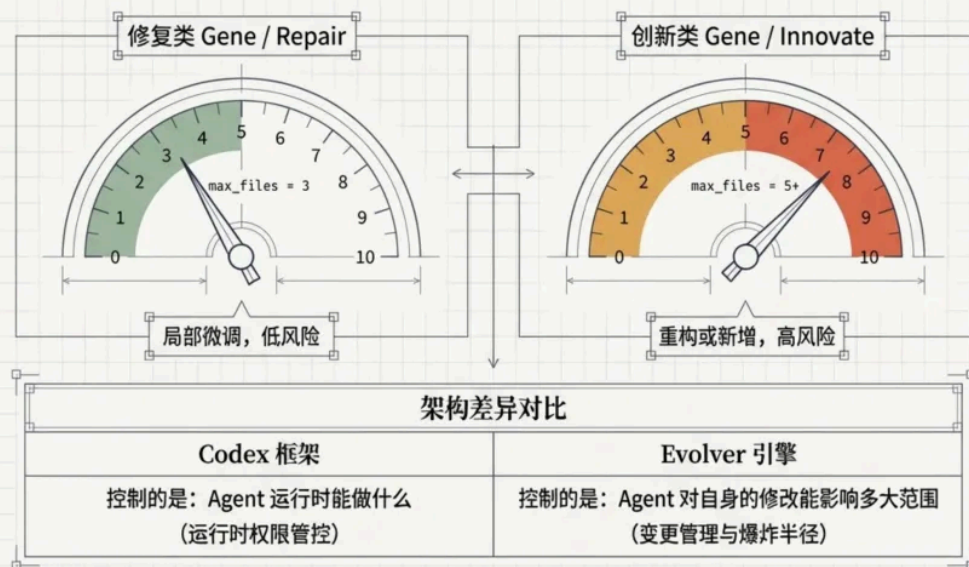
这不是简单的“上次成功就再用”。它是一个因果推理链：在特定信号条件下，用某个 Gene 生成某个 Mutation，执行后结果如何？下一次遇到相似信号，系统会跳过已经证明无效的策略，优先选成功率高的。

对比 OpenClaw 的会话历史（只记内容不记因果）、Codex 的推理轨迹保全（保模型的思维连续性不保跨 session 的策略学习），Evolver 的记忆图谱是我在这四个项目里看到的唯一一个在 harness 层面做了 **策略级因果学习** 的设计。

拆解三：爆炸半径控制与固化流程

`max_files` 这个约束值得说一下。每个 Gene 都定义了自己允许修改的最大文件数。一个修复错误的 Gene 可能限制在 3 个文件以内；一个创建新技能的 Gene 可能允许 5 个。这是编译期确定的爆炸半径约束，不依赖运行时判断。

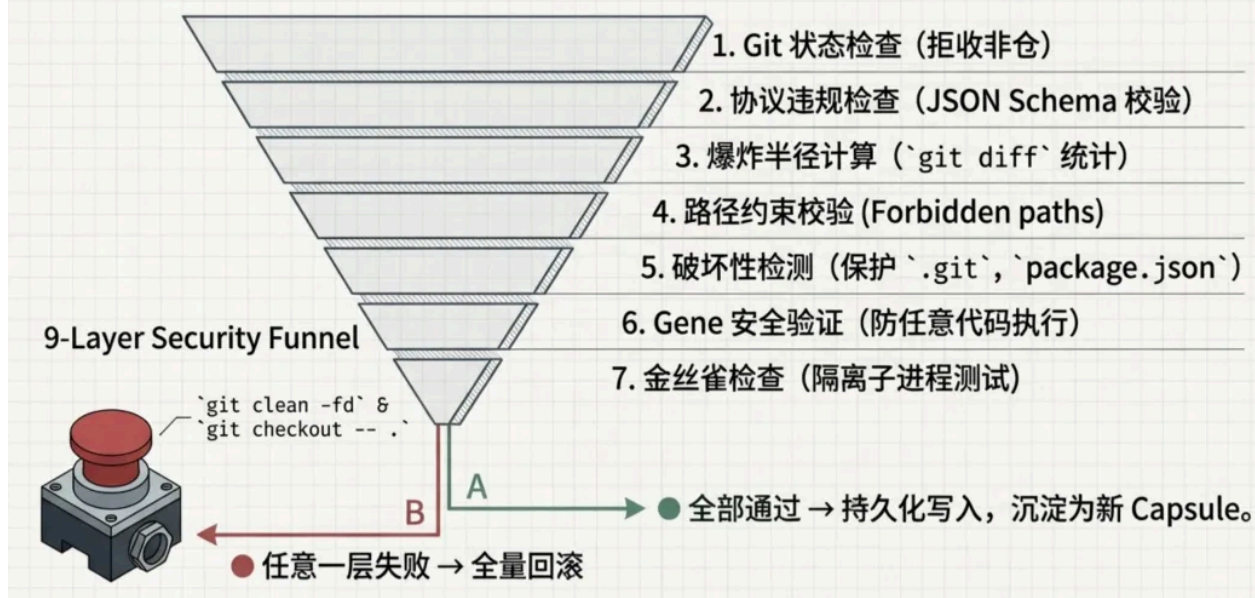
第三站：爆炸半径约束 (Blast Radius Control)



对比 [Codex](#) 的三层策略引擎 (Starlark + Guardian + 用户确认), Evolver 的安全模型完全不同。Codex 控制的是"Agent 当前能做什么操作", Evolver 控制的是"Agent 对自身的修改能影响多大范围"。一个是运行时权限控制, 一个是变更管理。

同时, Evolver 的固化流程 (solidify) 是整个系统的安全闸门:

第四站：Solidify 固化与安全闸门

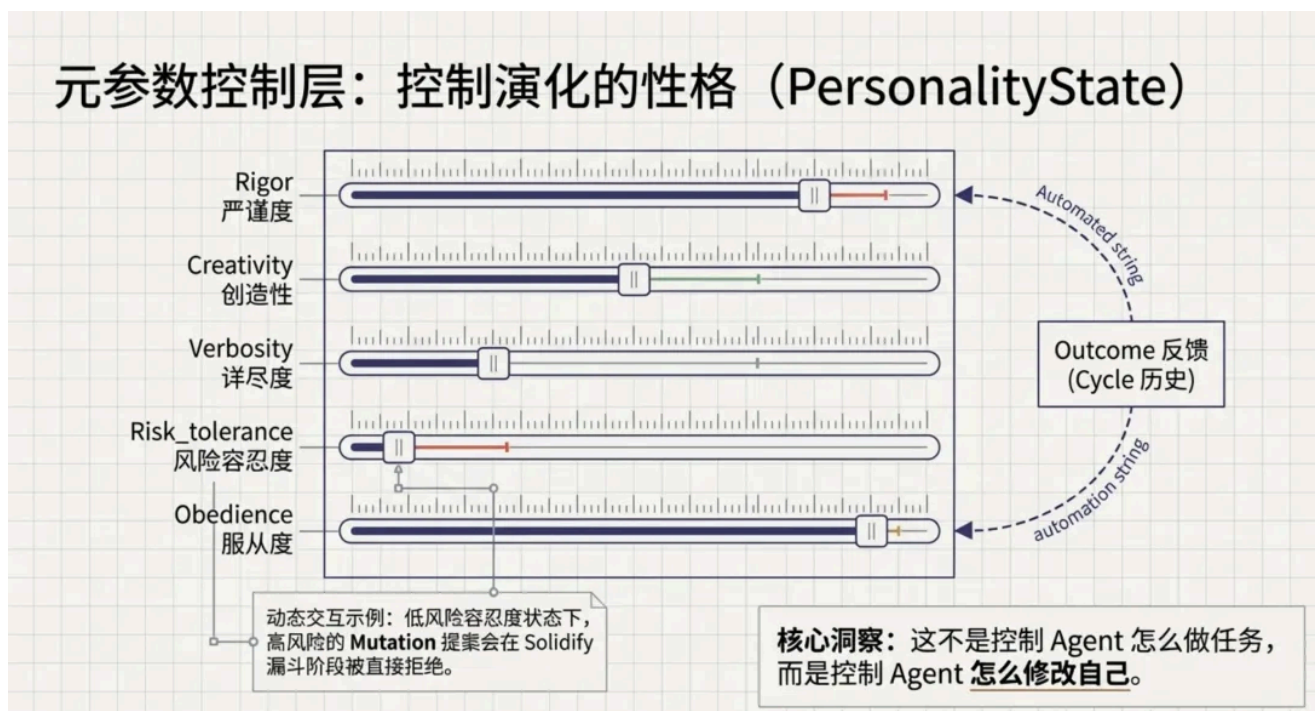


1. **Git 状态检查** ——不在 Git 仓库里就拒绝运行
2. **协议违规检查** ——Mutation 或 PersonalityState 对象缺失/格式错误就拒绝
3. **爆炸半径计算** ——用 git diff 统计变更的文件数和行数

4. **约束校验** ——对照 Gene 定义的 max_files、forbidden_paths 检查
5. **破坏性变更检测** ——改了 .git、package.json、核心依赖？直接回滚
6. **Gene 验证** ——执行 Gene 定义的验证命令（有严格的安全检查，防止任意代码执行）
7. **金丝雀检查** ——用一个隔离子进程验证 index.js 还能正常加载
8. **全部通过** → 持久化 EvolutionEvent、Capsule，更新 Gene
9. **任何一步失败** → git checkout -- . + git clean -fd，回滚所有变更

拆解四：人格参数与策略漂移

Evolver 有一个 PersonalityState，包含五个 0-1 之间的连续参数：



- rigor：严谨度
- creativity：创造性
- verbosity：输出详细程度
- risk_tolerance：风险容忍度
- obedience：服从度

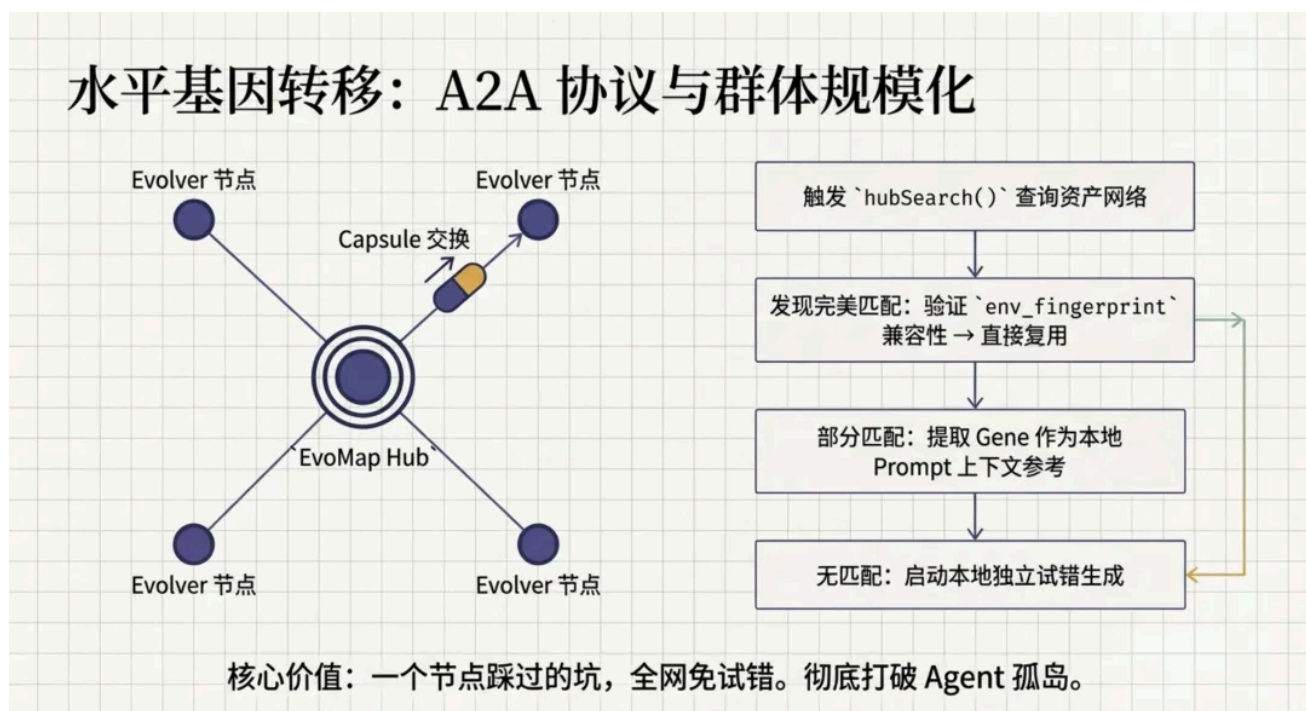
这些参数不是静态配置，会根据过去 cycle 的反馈演化。高风险 Mutation 在低 risk_tolerance 状态下会被 solidify 层直接拒绝。

mutationCategoryFromContext 根据信号和人格状态选择策略：有错误信号就 repair；没错误但有机会信号就 innovate；EVOLVE_STRATEGY 环境变量可以覆盖默认行为（balanced、innovate、harden、repair-only）。

说白了，PersonalityState 是一个对 Agent 自我改进行为的元参数控制层。它不控制 Agent 怎么完成用户任务，而是控制 Agent 怎么修改自己。

拆解五：A2A 协议与水平基因转移

Evolver 可以通过 A2A（Agent-to-Agent）协议连接到 EvoMap Hub，在多个 evolver 节点之间共享 Gene 和 Capsule。



工作流程：每次演化前先 hubSearch()——如果 Hub 上有完整的 Capsule 匹配当前信号，直接复用；有部分匹配的 Gene，作为 prompt 上下文参考；都没有，才走本地 Gene 选择。

外部资产不直接进本地 asset store。它要经过一个 staging → manual review → promotion 的流程，promotion 时会强制检查验证标记和安全命令。

这就是"水平基因转移"——一个节点发现的修复策略，可以被其他遇到相似问题的节点复用，不用每个节点都独立试错。Capsule 内嵌了 `env_fingerprint`，记录了这个策略在什么环境下被验证过，跨环境复用时可以评估兼容性。

回到之前聊的

读 Evolver 的架构时，我始终在想之前写过的 [《谷歌 AlphaEvolve：一场由 AI 主导的代码物竞天择》](#)。

AlphaEvolve 的核心循环是：LLM 生成代码变异 → 自动评估 → 进化数据库筛选优胜者 → 下一代进化。Evolver 的核心循环是：提取信号 → 选择 Gene → 生成 Mutation → 固化验证 → 成功的 Capsule 沉淀。两者的结构同源，但应用层级完全不同。

全局视角：前沿 Harness 范式对比矩阵

	OpenClaw / Codex	AlphaEvolve	Evolver
优化目标	把当前任务做好	发现更优的数学/算法代码	改进 Agent 自身的 Harness
评估函数	用户反馈 / 单元测试	强确定性 (算出或没算出)	模糊验证 (安全漏斗+验证命令)
记忆机制	内容与思维连续性记录	MAP-Elites 种群多样性数据库	策略级因果图谱

从单纯的“进化搜索”到严密的“因果学习”之跨越。

AlphaEvolve 进化的是算法代码——矩阵乘法、排序网络、数学难题的求解程序。它的评估函数（evaluator）是确定性的：代码要么算出正确答案，要么没有。进化搜索空间巨大但反馈信号清晰。

Evolver 进化的是 Agent 自身的 harness——prompt、技能定义、记忆文件、行为策略。它的评估依赖固化流程里的约束校验和 Gene 定义的验证命令，反馈信号远没有 AlphaEvolve 那么干净。一个 prompt 改动的效果可能要跑好几轮才能看出来。

但 Evolver 做了一件 AlphaEvolve 没做的事：记忆图谱。AlphaEvolve 靠进化数据库（MAP-Elites）维持种群多样性，但不记录“某个变异为什么失败”的因果链。Evolver 的 SignalSnapshot → Hypothesis → Attempt → Outcome 链条把每次演化的决策过程都记下来了，并且用这些记录指导后续的 Gene 选择。这是从“进化搜索”到“因果学习”的一步跨越。

我的判断

Evolver 的野心比前三个项目都大。OpenClaw、Codex、MimiClaw 都在做“让 Agent 把事做好”的 harness，Evolver 在做“让 harness 改进自己”的 harness。这是 meta-level 的工程。

信号提取 + Gene 匹配 + 固化验证这条流水线设计得很扎实。尤其是反停滞机制——信号压制、强制创新切换、失败 Gene 自动 ban——这些是真正跑过自演化系统的人才会想到的问题。没有这些机制，一个自演化 Agent 大概率会在 repair loop 里无限打转。

系统局限：天花板与未知风险 (System Constraints)

Warning	Warning
限制 1：幻觉闭环 (The Hallucination Loop)	限制 2：启发式漂移 (Heuristic Drift)
Gene 的提炼当前重度依赖 LLM 的自我评估能力。	PersonalityState (人格参数) 的连续演化缺乏类似 RL 的明确 Reward 梯度。
→ 隐患：一个 LLM 判断另一个 LLM 生成的策略是否值得复用，存在“盲人互捧”的自我幻觉风险。质量瓶颈受限于底层大模型。	→ 隐患：在缺乏确定性 Evaluator 的模糊创新任务中，可能导致系统长期陷入无梯度的启发式漂移，稳定性存疑。

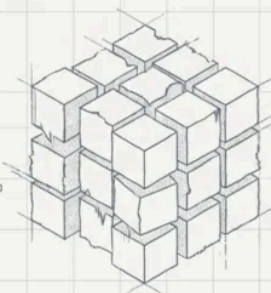
结论：在有明确信号的修复任务 (Repair) 上无敌，但在目标模糊的创新演化 (Innovate) 上充满风险。

但我有两个担心。第一，Gene 的质量决定了演化的天花板。当前的 Gene 是手动定义的 JSON 策略模板，虽然有 skill distillation 从成功 Capsule 中自动提炼新 Gene，但这个提炼过程本身依赖 LLM——一个 LLM 判断另一个 LLM 的策略是否值得复用，可靠性存疑。第二，PersonalityState 的五个参数在 0 到 1 之间连续变化，但调整策略没有明确的梯度信号——不像 RL 有 reward function，也不像

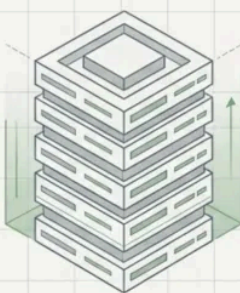
AlphaEvolve 有确定性 evaluator。人格参数的演化更像是启发式漂移，长期稳定性未知。

Cognition 在 Devin 的年度绩效报告里说了一句话：Devin 是"junior execution at infinite scale"——在有明确需求和可验证结果的任务上无限并行。Evolver 的 Gene + Capsule 体系也有类似的特征：它最可靠的场景是那些信号清晰、验证命令确定性强的修复类任务。越往 innovate 方向走，评估越模糊，风险越大。

核心启示：Agent 的后台维护 Daemon



现状态：Prompt 腐化
大多数 Agent 的崩溃源于缺乏持续的日志分析、模式提取和自动化调优。



未来态：Background Daemon
自动化日常维护。一个安静在后台运行、持续自修复的守护进程。

即使 Evolver 只做到了自动化 **Repair** 和 **Optimize**，它也解决了一个巨大的商业痛点：让 Agent 的自演化变成了一个“**跑着跑着自己就修好了**”的**后台服务**。

不过，即使只是把 repair 和 optimize 自动化了，Evolver 已经解决了一个实际问题：Agent 的日常维护。大多数 Agent 产品的 prompt 腐化不是因为哪个决策错了，而是因为没有人持续去看日志、提取模式、做调优。Evolver 让这件事变成了一个后台 daemon——跑着跑着自己就修了。在我的硬件 Agent 产品场景下，这件事本身就值得借鉴。

Reference

项目仓库

- [EvoMap/evolver](#)

文章参考

- Anthropic – "Effective Harnesses for Long-Running Agents"
- Cognition – "Devin's 2025 Performance Review"

过去文章

-  《谷歌AlphaEvolve：一场由AI主导的代码物竞天择》



航天智人

一个在熵增之海上航行的生命，想要理解这片熵增之海

102篇原创内容



公众号

Agent工程 · 目录 ≡

＜ 上一篇

MimiClaw 拆解：一个跑在 5 美元单片机上的
Agent Harness

下一篇 ＞

PicoClaw 拆解：一个 10MB 的 Go Agent
Runtime 和它的 AI 自举史